

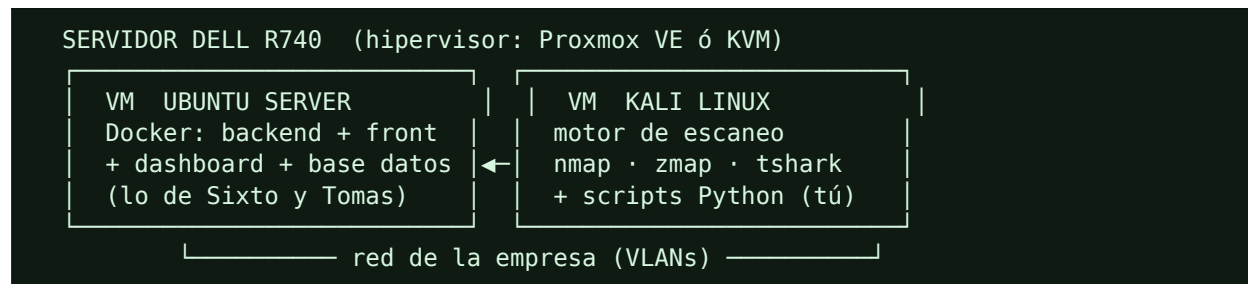
Sentinel // SOC

Kali en la Fase 1: función, conexión y tus tareas

Qué hace Kali, por qué se necesita, cómo se conecta con el dashboard, y qué te toca a ti

1. La foto completa — el servidor Dell R740

El R740 (2x Xeon Silver 4114, 128 GB RAM, 12 TB) es de sobra para todo. La forma correcta de meter Ubuntu Server y Kali en la misma máquina no es dualboot, sino un hipervisor con dos máquinas virtuales:



Con 128 GB de RAM caben las dos VMs holgadas y sobra para la Fase 2 (Suricata, más VMs). Recomendación de hipervisor: Proxmox VE (gratis, ideal para mezclar Linux/Windows) o KVM sobre el propio Ubuntu.

2. ¿Por qué se necesita Kali? (su función)

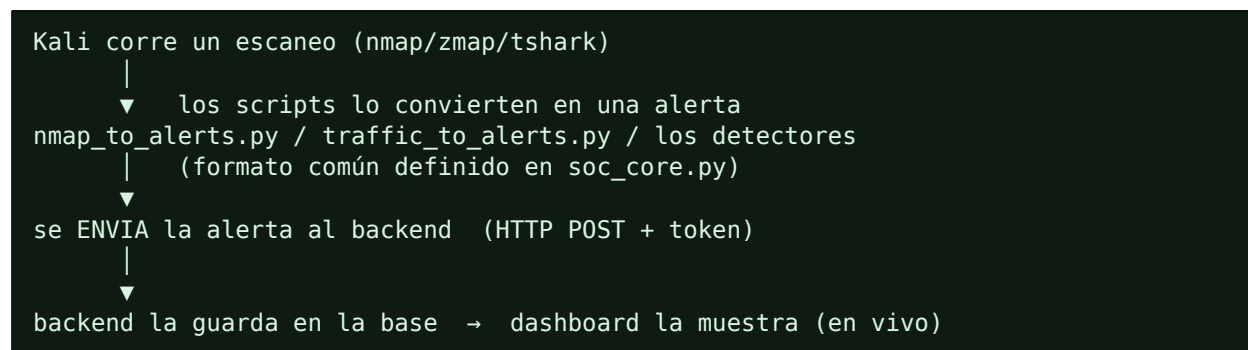
El dashboard + backend en Ubuntu es el cerebro y la cara: recibe, guarda y muestra las alertas, y maneja el login. Pero por sí mismo NO sale a la red a buscar nada. Alguien tiene que salir activamente a escanear y detectar — ese es Kali.

Kali = el motor de escaneo / sensor. Corre nmap y zmap (puertos, servicios, vulnerabilidades), tshark (tráfico — más para Fase 2) y los detectores de Python (intrusos, phishing, malware). Todo eso genera las alertas.

Honestidad técnica: no es que tenga que ser Kali por magia; se necesita un host Linux con las herramientas y con alcance a la red. Kali es ese host (trae todo listo, es tu entorno de seguridad, y en Fase 2 será tu caja de pentesting). Separarlo del backend además es más seguro. En una frase: Ubuntu muestra, Kali detecta.

3. Cómo se conecta Kali con Python, los scripts y el dashboard

Los scripts VIVEN en Kali (en /opt/sentinel-soc). El flujo completo es:



El punto de conexión clave: hoy los scripts escriben la alerta a un archivo local (data/alerts.json). Para el sistema real hay que agregar un “forwarder” que haga POST al endpoint de ingesta del backend. Eso lo defines CON Sixto y Tomás: ellos te dan la URL del endpoint y el token; tú conectas la salida de los scripts. (Puedo escribirte ese forwarder en cuanto tengas el endpoint.)

4. Aclaración sobre era.ca y la red

era.ca es el dominio de Google Workspace de la empresa; sirve para el LOGIN del dashboard (SSO). Kali NO necesita “unirse” a ese dominio. Lo que Kali sí necesita:

- Estar en la red con red bridged e IP estática (no NATeada — ese fue el problema del Docker).
- Alcance a las VLANs que va a escanear (VLAN IDs / trunk que coordina IT).

Tus compañeros necesitan la IP del servidor + la VLAN para llegar al dashboard; tú además necesitas que el Kali alcance lo que va a escanear.

5. De dónde sale cada tipo de alerta (clave para que detecte “todo eso”)

Los escaneos de red son automáticos desde Kali, pero intrusos/phishing/malware necesitan que les llegue la fuente de datos. Esto lo coordinas con IT y los devs:

Tipo de alerta	De dónde salen los datos
Puertos abiertos / vulnerabilidades	nmap y zmap desde Kali — directo, ya lo tienes.
Intrusos / logins del dominio	Los LOGS: auth.log de servidores Linux (SSH) y/o el Security log de Windows (Domain Controller). Hay que hacer que esos logs lleguen a Kali (syslog remoto o export). login_monitor.py los lee.
Correos de phishing	Una carpeta/buzón donde se depositen los .eml sospechosos (reportados por usuarios o exportados del correo). phishing_detector.py los analiza.
Malware	Archivos/hasheos de una carpeta de cuarentena. malware_detector.py los revisa.

6. Tus tareas paso a paso (Fase 1)

1. Desplegar la VM Kali en el server (la imagen que armaste): red bridged, IP estática, acceso a las VLANs autorizadas.
2. Verificar la personalización (setup-kali-soc.sh ya corrió): herramientas, cuentas admin/dev, SSH.
3. Cargar targets.conf con el alcance autorizado (scan_scope).
4. Conectar la ingesta al backend: el forwarder que hace POST de cada alerta al endpoint (con Sixto y Tomás).
5. Programar los escaneos 24/7 con cron o systemd timers (descubrimiento + puertos seguido; vulnerabilidades en la ventana coordinada con IT).
6. Dejar el login_monitor.py como servicio systemd (--follow) y conectar las fuentes de log (auth.log / Security log).
7. Prueba de punta a punta: un escaneo real genera una alerta que llega al dashboard en vivo.
8. QA de seguridad (probar que la reja scan_scope rechaza lo no autorizado; auditar el hardening) y producir el reporte de línea base (Fase A).

7. Qué es Fase 2 (no ahora)

Deja el Kali listo pero enfócate en la Fase 1. La Fase 2 incluirá: Suricata (IDS) y monitoreo de tráfico continuo, pentesting activo, “ataques” mensuales controlados para medir vulnerabilidades, y la revisión de arquitectura de red y firewalls para mejorar la seguridad. Ahí es donde tshark y las herramientas ofensivas de Kali entran de lleno.



Todo escaneo, en Fase 1 y Fase 2, solo sobre lo autorizado por escrito y en la ventana coordinada con IT.

Resumen en una frase

Los devs dejan corriendo el dashboard y el backend en la VM Ubuntu; tú dejas corriendo la VM Kali que los alimenta (escaneos programados + detectores + envío de alertas al backend) y auditas que todo el conjunto sea seguro. Ubuntu muestra, Kali detecta.